

InsightFlow Agent 应用场景文档

1. 文档目标

本文档用于定义 InsightFlow Agent 的核心应用场景和 Demo 展示方式。

InsightFlow Agent 的核心定位是：

Multi-Agent Tool-Calling BI Workflow

因此，应用场景不能只是“问数据、答数据”，而要能展示：

不同 Agent 分工协作
多个工具真实调用
SQL 执行反馈
SQL 错误修复
安全校验
Trace
Eval
报告生成
后续行动型 workflow

2. 场景优先级总览

阶段	场景	目标
P0	SQL 错误自动修复	证明不是普通 Text2SQL
P0	Metric-aware 销售额查询	证明有业务指标口径
P0	危险 SQL / 敏感字段拦截	证明工具调用有安全边界
P1	销售下降分析报告	证明可生成可追溯报告
P2	经营周报生成	证明可做多模块业务报告
P2	销售下降行动闭环	证明可从分析走向行动

S1：SQL 错误自动修复

1. 场景定位

这是 P0 最重要的技术 Demo。

它用于证明系统不是普通 Text2SQL，而是具备：

```
Action
Observation
Revision
Retry
Trace
```

的 Agentic SQL Workflow。

2. 用户输入

最近 30 天销售额最高的 5 个商品是什么？

3. 故意制造的错误 SQL

SQL Generator 第一次可能生成：

```
SELECT
  p.product_name,
  SUM(oi.quantity * oi.price) AS sales
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.status = 'paid'
  AND o.order_date >= date('now', '-30 days')
GROUP BY p.product_name
ORDER BY sales DESC
LIMIT 5;
```

问题：

order_items 表中不存在 price 字段，正确字段是 unit_price。

4. Agent 执行流程

```
Schema Agent
-> get_database_schema()

Metric Agent
```

-> retrieve_metric_definition()

SQL Generator Agent

-> 生成初始 SQL

SQL Reviewer Agent

-> validate_sql()

SQL Executor

-> run_sql()

-> 返回 no such column: oi.price

Error Fix Agent

-> 根据 error + schema 修复 SQL

SQL Reviewer Agent

-> validate_sql()

SQL Executor

-> run_sql()

Insight Agent

-> 生成最终回答

Trace Logger

-> log_trace()

5. 修复后 SQL

```
SELECT
  p.product_name,
  SUM(oi.quantity * oi.unit_price) AS sales
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.status = 'paid'
  AND o.order_date >= date('now', '-30 days')
GROUP BY p.product_name
ORDER BY sales DESC
LIMIT 5;
```

6. 展示重点

真实 SQL 执行
数据库错误反馈

```
Error Fix Agent
重新 validate_sql
重新 run_sql
最多 retry 一次
Trace 记录完整修复路径
```

7. 输出产物

```
generated_sql
execution_error
fixed_sql
fix_reason
review_result
execution_result
final_answer
trace.json
```

S2: Metric-aware 销售额查询

1. 场景定位

该场景用于证明系统不是只根据 Schema 生成 SQL，而是会先读取业务指标口径。

2. 用户输入

最近 30 天销售额最高的 5 个商品是什么？

3. 业务指标定义

Metric Agent 调用：

```
retrieve_metric_definition("销售额")
```

返回：

```
gmv:
  formula: "SUM(order_items.quantity * order_items.unit_price)"
```

```
required_filters:  
- "orders.status = 'paid'"
```

4. Agent 执行流程

```
Metric Agent  
-> retrieve_metric_definition()  
  
Schema Agent  
-> get_database_schema()  
  
SQL Generator Agent  
-> 使用 GMV 口径生成 SQL  
  
SQL Reviewer Agent  
-> 检查是否使用 unit_price  
-> 检查是否包含 orders.status = 'paid'  
  
SQL Executor  
-> run_sql()  
  
Insight Agent  
-> 生成回答
```

5. 预期输出

```
最近 30 天销售额最高的 5 个商品分别是：  
1. Camera A: 128,000  
2. Lens B: 113,500  
3. Headphone C: 98,000  
4. Tripod D: 87,500  
5. Keyboard E: 79,600
```

6. 展示重点

```
Metric Agent  
Metric Tool  
业务指标定义  
SQL Reviewer 口径校验  
不是普通 Text2SQL
```

S3: 危险 SQL / 敏感字段拦截

1. 场景定位

该场景用于展示工具调用安全边界。

Agent 不是生成什么 SQL 都执行，而是必须经过 `validate_sql()`。

2. 危险 SQL 输入示例

删除所有取消订单的数据。

SQL Generator 可能生成：

```
DELETE FROM orders WHERE status = 'cancelled';
```

SQL Reviewer 必须拒绝。

3. 敏感字段输入示例

帮我导出所有用户的手机号和邮箱。

SQL Generator 可能生成：

```
SELECT name, phone, email FROM users LIMIT 100;
```

SQL Reviewer 必须拦截敏感字段。

4. Agent 执行流程

SQL Generator Agent

-> 生成 SQL

SQL Reviewer Agent

-> `validate_sql()`

-> 发现非 SELECT 或敏感字段

Workflow

-> 不进入 `run_sql`

Insight Agent
-> 返回拒绝原因

Trace Logger
-> 记录 blocked query

5. 预期输出

该查询涉及敏感字段 phone 和 email，当前系统不允许导出直接用户联系方式。
系统已拒绝执行 SQL。

你可以改为查询城市级用户数量、用户分布或聚合统计结果。

6. 展示重点

SQL Reviewer Agent
validate_sql()
工具调用安全边界
敏感字段拦截
不进入 run_sql
Trace 记录

S4：销售下降分析报告

1. 场景定位

这是 P1 主 Demo。

它用于展示系统不只是查 SQL，还能生成可追溯的业务分析报告。

2. 用户输入

最近 3 个月销售额下降最多的品类有哪些？生成趋势图和分析报告。

3. Agent 执行流程

Supervisor Agent

-> 判断为 decline_analysis_report

Schema Agent

-> get_database_schema()

Metric Agent

-> retrieve_metric_definition()

Context Retrieval Agent

-> retrieve_business_context()

SQL Generator Agent

-> 生成品类月度 GMV 查询 SQL

SQL Reviewer Agent

-> validate_sql()

SQL Executor

-> run_sql()

Python Analyst

-> run_python_analysis()

-> 计算下降幅度、订单量变化、AOV 变化

Evidence Validator Agent

-> 区分数据支持结论和假设

Chart Agent

-> generate_chart()

Report Agent

-> save_report()

Trace Logger

-> log_trace()

4. 预期输出

reports/charts/run_001_category_gmv_trend.png

reports/markdown/run_001_decline_analysis.md

logs/traces/run_001.json

5. 报告内容

用户问题
使用的指标定义
执行 SQL
查询结果摘要
数据支持结论
需要进一步验证的假设
图表路径
下一步建议
Trace 路径

6. 数据支持结论示例

Camera 品类最近 3 个月 GMV 下降 32.8%。
Camera 品类订单量下降 29%，AOV 仅下降 5.2%。
因此，当前数据支持“下降主要来自订单量下降”。

7. 需要进一步验证的假设示例

可能与广告流量下降有关，但当前数据库没有曝光、点击率和转化率数据，无法验证。
可能与库存不足有关，但当前数据库没有库存字段，无法验证。

8. 展示重点

多 Agent 协作
多工具调用
SQL 执行结果驱动分析
Evidence Validator 防止胡说
图表和报告是真实文件
报告结论可追溯

S5：经营周报生成

1. 场景定位

这是 P2 的报告型 Demo。

它用于展示系统可以把多个分析子任务组合成经营周报。

注意：它不是 P0 功能。

2. 用户输入

帮我生成一份本周电商经营分析周报，包括销售额、订单量、Top 商品、下降品类、渠道表现和运营建议。

3. Report Supervisor 拆解任务

本周 GMV
本周订单量
本周客单价
Top 5 商品
Top 5 品类
下降最多品类
渠道表现
异常点
下周建议

4. Agent 执行流程

```
Report Supervisor Agent
-> 拆解周报任务

Schema Agent
-> get_database_schema()

Metric Agent
-> retrieve_metric_definition()

SQL Generator Agent
-> 为多个模块分别生成 SQL

SQL Reviewer Agent
-> validate_sql()

SQL Executor
-> run_sql()

Python Analyst
-> run_python_analysis()
```

Evidence Validator

-> 校验核心结论

Chart Agent

-> generate_chart()

Report Agent

-> save_report()

5. 输出产物

```
reports/markdown/weekly_business_report.md
reports/charts/weekly_sales_trend.png
reports/charts/top_products.png
reports/charts/category_decline.png
reports/data/weekly_metrics.csv
logs/traces/run_xxx.json
```

6. 周报结构

1. 核心结论
2. 核心指标
 - GMV
 - 订单量
 - 客单价
 - 环比变化
3. Top 商品
4. Top 品类
5. 销售下降品类
6. 渠道表现
7. 数据支持结论
8. 需要进一步验证的假设
9. 下周建议
10. Trace 与 SQL 记录

7. 展示重点

一个用户请求触发多个 SQL 子任务
多个工具调用组合成报告

报告不是 LLM 直接编写
所有关键结论可追溯到 SQL 和 execution_result

S6：销售下降行动闭环

1. 场景定位

这是 P2 的行动型 Agent Demo。

它用于回应 “InsightFlow 是否行动能力不如 RepoPilot” 的质疑。

InsightFlow 的行动对象不是代码仓库，而是业务运营 workflow。

2. 用户输入

找出最近销售额下降最多的品类，并为运营团队创建跟进任务和监控告警。

3. Agent 执行流程

```
Schema Agent
-> get_database_schema()

Metric Agent
-> retrieve_metric_definition()

SQL Generator Agent
-> 生成下降分析 SQL

SQL Reviewer Agent
-> validate_sql()

SQL Executor
-> run_sql()

Evidence Validator
-> 生成数据支持结论

Action Planner
-> 生成 action_plan

Risk Assessor
-> 判断 create_task / create_metric_alert 需要审批
```

Approval Gate

-> 等待人工审批

Action Executor

-> create_task()

-> create_metric_alert()

Action Verifier

-> verify_action_execution()

Audit Logger

-> log_audit_event()

4. 行动计划示例

```
{
  "action_plan": [
    {
      "action_type": "create_task",
      "title": "排查 Camera 品类订单量下降原因",
      "reason": "Camera 品类 GMV 下降 32.8%，订单量下降 29%",
      "priority": "high",
      "owner": "ops_team",
      "due_date": "2026-06-25"
    },
    {
      "action_type": "create_metric_alert",
      "metric": "category_gmv_drop_rate",
      "target": "Camera",
      "condition": "drop_rate > 20%",
      "window": "7d",
      "severity": "medium"
    }
  ]
}
```

5. 审批请求示例

```
{
  "run_id": "run_001",
  "pending_actions": ["create_task", "create_metric_alert"],
  "summary": "将为 Camera 品类创建 1 个运营排查任务和 1 个 GMV 下降监控告警。",
  "risk_level": "medium",
}
```

```
"approval_required": true  
}
```

6. 输出产物

```
action_plan.json  
approval record  
tasks record  
metric_alerts record  
action_verification_result.json  
audit_logs record  
trace.json
```

7. 展示重点

```
Action Planner Agent  
Risk Assessor Agent  
Approval Gate  
create_task()  
create_metric_alert()  
Action Verifier  
Audit Log  
业务行动型 workflow
```

3. Demo 推荐顺序

3.1 P0 面试 Demo 顺序

第一优先级：

```
S1: SQL 错误自动修复
```

第二优先级：

```
S2: Metric-aware 销售额查询
```

第三优先级：

S3: 危险 SQL / 敏感字段拦截

3.2 P1 面试 Demo 顺序

S4: 销售下降分析报告

重点展示:

Evidence Validator
Chart Agent
Report Agent
save_report()

3.3 P2 面试 Demo 顺序

S5: 经营周报生成
S6: 销售下降行动闭环

重点展示:

多 SQL 子任务
报告交付物
Action Planner
Approval Gate
Action Tool Calling

4. 场景与能力映射

场景	Multi-Agent	Tool Calling	SQL Execution	Error Repair	Report	Action
S1 SQL 错误修复	是	是	是	是	否	否
S2 Metric-aware 查询	是	是	是	可选	否	否
S3 安全拦截	是	是	否	否	否	否
S4 销售下降报告	是	是	是	可选	是	否

场景	Multi-Agent	Tool Calling	SQL Execution	Error Repair	Report	Action
S5 经营周报	是	是	是	可选	是	可选
S6 行动闭环	是	是	是	可选	是	是

5. 最终场景策略

不要一开始做 8 个场景。

最终策略：

P0 做 3 个核心场景：

1. SQL 错误自动修复
2. Metric-aware 查询
3. 安全拦截

P1 做 1 个高质量报告场景：

4. 销售下降分析报告

P2 做 2 个升级场景：

5. 经营周报生成
6. 销售下降行动闭环

这样既能收敛开发复杂度，又能保持项目上限。

6. 项目场景一句话总结

InsightFlow 的核心场景不是普通数据问答，而是：

用户提出业务问题后，多个 Agent 协作调用真实工具完成 SQL 生成、校验、执行、错误修复、数据解释、报告生成和后续行动。

P0 证明：

它会调用工具并根据工具反馈修复。

P1 证明：

它能生成可靠、可追溯的数据分析报告。

P2 证明：

它能把数据洞察转化为业务行动。